

Chapter 1

Interpretable regression with checked physical deployment

1.1 An inspectable model of component transformations

A component prediction is useful for engineering analysis when its dependence on the inputs can be examined and its material accounting can be checked. These requirements address different questions. A transparent regression equation explains how the predicted state depends on operating conditions and influent loads. A conservation test determines whether that state preserves the inventories represented by the reaction network. A nonnegativity test determines whether each predicted concentration is physically meaningful. An explicit equation can fail either physical test. A physically corrected prediction can also remain difficult to interpret if the underlying regression is opaque.

Invariant-constrained second-order regression (ICSOR) combines an explicit polynomial driver, learned dependence among effluent components, and checked deployment. The driver retains separate terms for operating conditions, influent composition, curvature, and interactions. The coupling relation expresses how the driver contributes to the predicted component state. Deployment then checks that state and applies a constrained correction when necessary. The interpretation of the regression and the guarantee provided by deployment therefore remain distinct.

The preference for an inspectable regression follows the argument of ? that transparent model structures can support decisions that require scrutiny. It also follows the broader development of models that combine learned relations with physical structure discussed by ?. Transparency does not establish that a fitted coefficient is a biological mechanism. It makes the relation available for examination, comparison with process knowledge, and testing across operating conditions.

Activated sludge nutrient removal provides a strong reason to retain interactions.

Carbon availability influences denitrification and the activity of organisms that store phosphorus. Oxygen availability influences both carbon oxidation and nitrification. ? describe these connected pathways, and ? examine the role of carbon source in the competition between phosphorus-storing organisms and denitrifiers. A regression that represents an aeration effect independently of loading can overlook such dependencies. An interaction term can describe an association between an operating action and a loading condition while keeping that association visible.

The prediction target is the effluent component vector $c_{out} \in \mathbb{R}^F$. The adopted process basis has $F = 20$ components and $P = 28$ reactions. The operating vector $u \in \mathbb{R}^{M_{op}}$ has $M_{op} = 2$ entries for HRT and aeration level. The full influent vector is $c_{in} \in \mathbb{R}^F$. Component concentrations retain their physical units. The four reported composites are obtained from a fixed map $I_{comp} \in \mathbb{R}^{4 \times F}$. That map is applied after component prediction.

1.2 Conservation in the adopted state space

The link between a component balance and a regression constraint can first be seen in a hypothetical conversion of one component into another. Suppose that one unit of component 1 is converted into one unit of component 2. The two components are expressed in a common inventory unit for this illustration. Their changes satisfy

$$c_{out,1} - c_{in,1} = -\xi, \quad c_{out,2} - c_{in,2} = \xi. \quad (1.1)$$

Adding these equations eliminates the unknown conversion amount. The relation available to the regression is $c_{out,1} + c_{out,2} = c_{in,1} + c_{in,2}$. With an illustrative influent $(6, 4)^\top$ and conversion amount 2, the effluent is $(4, 6)^\top$. Both totals are 10. Many other non-negative pairs also have that total. The balance therefore limits the possible regression output without identifying the particular state produced by the kinetics.

The same calculation can be written using the small matrices

$$\nu = \begin{bmatrix} -1 & 1 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad A\nu^\top = 1(-1) + 1(1) = 0. \quad (1.2)$$

These matrices describe the reduced illustration, rather than the 20-component process. All numerical teaching examples use hypothetical coefficients and states. Their dimensions are reduced to make each operation visible, and they report no process observations. The adopted reactor retains its stated component basis, reaction matrix, and operating domain.

The stoichiometric matrix $\nu \in \mathbb{R}^{P \times F}$ records reactions as rows and components as columns. Let $V_0 \in \mathbb{R}^{F \times K}$ span its right null space, where $K = 5$. The invariant operator is

$$A = V_0^\top \in \mathbb{R}^{K \times F}, \quad \nu V_0 = 0, \quad A\nu^\top = 0. \quad (1.3)$$

The columns of V_0 are independent, so A has full row rank. With an orthonormal null-space basis, $AA^\top = I_K$. A different full-rank basis describes the same conservation conditions, although its row scaling changes the numerical size of a soft invariant penalty.

For the control volume and boundary accounting represented by the adopted process model, a reaction-induced change has the form

$$c_{out} - c_{in} = \nu^\top \xi, \quad \xi \in \mathbb{R}^P. \quad (1.4)$$

The reaction-progress vector ξ expresses the net contribution of the reactions in concentration-equivalent units. Multiplication by A removes that unknown contribution and gives

$$A(c_{out} - c_{in}) = 0. \quad (1.5)$$

This construction uses the conservation logic of the photochemical surrogate framework of ?. ? further show how a stoichiometric transformation can connect learned reaction information to conservative state changes. The same linear algebra applies here, but the conserved combinations are those of the activated sludge reaction network.

The equality in Equation (1.5) applies only when the modeled boundary supports the change relation in Equation (1.4). External gas transfer, chemical dosing, solids withdrawal, or another boundary flux must be included in the balance where it occurs. If an explicit boundary contribution s is present, the relevant relation becomes $A(c_{out} - c_{in} - s) = 0$. Using $Ac_{out} = Ac_{in}$ while omitting a boundary contribution would impose the wrong constraint. The invariant operator does not make an incomplete material balance correct.

For a new influent state, the component set used by checked deployment is

$$\mathcal{F}(c_{in}) = \{c \in \mathbb{R}^F \mid Ac = Ac_{in}, c \geq \mathbf{0}\}. \quad (1.6)$$

When $c_{in} \geq \mathbf{0}$, this set contains c_{in} and is therefore nonempty. Its members satisfy the modeled linear invariants and component nonnegativity. Membership does not show that a state solves the nonlinear kinetic equations, is dynamically stable, or is reachable at the stated operating point. The set gives necessary physical conditions within the declared boundary.

The reported composites provide another reason to work in component space. Distinct component states can have the same COD, TN, TP, and TSS values. A negative component can also be hidden by positive contributions to an aggregate. When I_{comp} is entrywise nonnegative, a nonnegative component state produces nonnegative composites. The reverse implication does not generally hold. Conservation and nonnegativity are therefore tested before the component state is collapsed into the four reported quantities.

1.3 A partitioned second-order driver

1.3.1 Separating operation from loading

Operating conditions and influent composition play different engineering roles. HRT and aeration define aspects of the reactor environment. Influent components define the material presented to that environment. This distinction is consistent with the operating-sensitivity analysis of ? and the separation of operational and influent uncertainty examined by ?.

Start with two hypothetical operating entries u_1, u_2 and two influent entries c_1, c_2 . A second-order model uses the individual entries, their squares, and their pairwise products. The operating products are $u_1^2, u_1u_2, u_2u_1, u_2^2$. Keeping both orders is an accounting convention for the feature vector. It does not make u_1u_2 different from u_2u_1 .

For two column vectors, the Kronecker operation multiplies the entire second vector by the first entry of the first vector, then by its second entry. Thus

$$\begin{aligned} u \otimes u &= \begin{bmatrix} u_1u_1 \\ u_1u_2 \\ u_2u_1 \\ u_2u_2 \end{bmatrix}, \quad c \otimes c = \begin{bmatrix} c_1c_1 \\ c_1c_2 \\ c_2c_1 \\ c_2c_2 \end{bmatrix}, \\ u \otimes c &= \begin{bmatrix} u_1c_1 \\ u_1c_2 \\ u_2c_1 \\ u_2c_2 \end{bmatrix}. \end{aligned} \tag{1.7}$$

For the illustrative values $u = (2, 3)^\top$ and $c = (4, 5)^\top$, these three blocks are respectively $(4, 6, 6, 9)^\top$, $(16, 20, 20, 25)^\top$, and $(8, 10, 12, 15)^\top$. Including the baseline 1 and the four first-order entries gives 17 ordered features in this reduced example. The same construction applies to the larger adopted input vector.

The feature vector retains both roles through the ordered expansion

$$\phi(u, c_{in}) = \begin{bmatrix} 1 \\ u \\ c_{in} \\ u \otimes u \\ c_{in} \otimes c_{in} \\ u \otimes c_{in} \end{bmatrix} \in \mathbb{R}^D, \quad D = 1 + M_{op} + F + M_{op}^2 + F^2 + M_{op}F. \tag{1.8}$$

The symbol $\otimes$ denotes the Kronecker product. The adopted dimensions give $D = 467$. The ordered expansion includes both u_iu_j and u_ju_i , and both $c_{in,i}c_{in,j}$ and $c_{in,j}c_{in,i}$. These pairs are equal as numerical features. Their interpretation requires the symmetry convention developed below.

Multiplication by regression coefficients can also be read one entry at a time. For an illustrative first driver, take the baseline coefficient as 1, operating coefficients as $(2, -1)$, and influent coefficients as $(0.5, 0)$. Let its operating quadratic row be $(0.1, 0.1, 0.1, 0)$, influent quadratic row be $(0, 0, 0, -0.05)$, and operating-load row be $(0, 0, 0.25, 0)$. Applying these rows to the numerical feature blocks gives

$$\begin{aligned}
r_1 &= 1 + \begin{bmatrix} 2 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 0.5 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \end{bmatrix} \\
&\quad + \begin{bmatrix} 0.1 & 0.1 & 0.1 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ 6 \\ 6 \\ 9 \end{bmatrix} \\
&\quad + \begin{bmatrix} 0 & 0 & 0 & -0.05 \end{bmatrix} \begin{bmatrix} 16 \\ 20 \\ 20 \\ 25 \end{bmatrix} \\
&\quad + \begin{bmatrix} 0 & 0 & 0.25 & 0 \end{bmatrix} \begin{bmatrix} 8 \\ 10 \\ 12 \\ 15 \end{bmatrix} \\
&= 1 + 1 + 2 + 1.6 - 1.25 + 3 = 7.35.
\end{aligned} \tag{1.9}$$

Each row-vector multiplication is a sum of coefficient times feature. It is the same operation as an ordinary linear regression, but the supplied features include nonlinear products. A second hypothetical driver $r_2 = -u_1 + u_2 + 0.2c_2$ would equal 2 at the same inputs. The two driver values need not be predicted concentrations until the coupled relation is solved.

Before grouping the coefficients into matrices, the first driver in this illustration can be written as

$$r_1 = 1 + 2u_1 - u_2 + 0.5c_1 + 0.1u_1^2 + 0.2u_1u_2 - 0.05c_2^2 + 0.25u_2c_1. \tag{1.10}$$

The coefficient 0.2 of u_1u_2 is the sum of the two ordered coefficients 0.1. In a general row, the scalar calculation is $r_f = \sum_{\ell=1}^D B_{f\ell}\phi_\ell$. Carrying out this sum for all components gives the driver $r \in \mathbb{R}^F$ through

$$r(u, c_{in}) = B\phi(u, c_{in}), \quad B \in \mathbb{R}^{F \times D}. \tag{1.11}$$

The driver is an intermediate regression quantity. It is not a vector of mechanistic reaction rates and is not the final deployed state. Its coefficients have no sign restriction, so the regression can represent both increasing and decreasing effects. Nonnegativity

is imposed on component states where it has a physical meaning. The driver block form is

$$\begin{aligned} r(u, c_{in}) = & b + W_u u + W_{in} c_{in} + \Theta_{uu}(u \otimes u) \\ & + \Theta_{cc}(c_{in} \otimes c_{in}) + \Theta_{uc}(u \otimes c_{in}). \end{aligned} \quad (1.12)$$

Here $b \in \mathbb{R}^F$ is a baseline term. The matrices $W_u \in \mathbb{R}^{F \times M_{op}}$ and $W_{in} \in \mathbb{R}^{F \times F}$ contain first-order operating and influent terms. The matrices $\Theta_{uu} \in \mathbb{R}^{F \times M_{op}^2}$ and $\Theta_{cc} \in \mathbb{R}^{F \times F^2}$ contain curvature within each input group. The matrix $\Theta_{uc} \in \mathbb{R}^{F \times M_{op} F}$ contains operating-load interactions.

1.3.2 Illustrative operating and loading effects

Consider an illustrative driver for one component with aeration level a and influent ammonium concentration n . A term of the form

$$r_f = \beta_0 + \beta_a a + \beta_n n + \beta_{aa} a^2 + \beta_{an} a n \quad (1.13)$$

has an aeration derivative $\partial r_f / \partial a = \beta_a + 2\beta_{aa} a + \beta_{an} n$. The effect described by this derivative changes with both aeration level and ammonium loading. This example uses a hypothetical polynomial and reports no fitted relation. It explains why a single linear aeration coefficient cannot summarize the response represented by the entire polynomial.

The same reasoning applies to simultaneous influent loads. A carbon-ammonium interaction allows the modeled association with carbon availability to depend on nitrogen loading. A phosphorus-storage interaction allows the modeled association with dissolved phosphate to depend on a particulate storage pool. These terms provide a transparent approximation to a nonlinear response. They do not replace the microbial pathways described by ?, and their signs do not establish causation.

The choice of a second-order library limits the model deliberately. It can represent smooth local curvature and pairwise input interactions. It cannot reproduce every threshold, saturation relation, or higher-order interaction exactly. Explicit nonlinear libraries are useful for inspecting candidate structure, as illustrated by ?. Their usefulness depends on the sampling domain, the adequacy of the library, and the stability of the fitted terms.

1.3.3 Symmetry and the meaning of cross terms

For a two-entry vector, direct multiplication gives

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = q_{11} x_1^2 + (q_{12} + q_{21}) x_1 x_2 + q_{22} x_2^2. \quad (1.14)$$

Only the sum of the two off-diagonal coefficients enters this polynomial. Replacing them by their average leaves that sum unchanged. For example,

$$Q = \begin{bmatrix} 0.1 & 0.3 \\ -0.1 & 0 \end{bmatrix}, \quad \frac{Q + Q^\top}{2} = \begin{bmatrix} 0.1 & 0.1 \\ 0.1 & 0 \end{bmatrix}. \quad (1.15)$$

At $x = (2, 3)^\top$, the first matrix gives $0.4 + 1.8 - 0.6 = 1.6$. The symmetric matrix gives $0.4 + 0.6 + 0.6 = 1.6$. The asymmetric coefficients cannot be distinguished from these predictions. Their squared sums are different. The original matrix has squared sum $0.01 + 0.09 + 0.01 = 0.11$, while its symmetric part has squared sum $0.01 + 0.01 + 0.01 = 0.03$. The removed part has squared sum 0.08 and contributes no prediction. This is why a positive coefficient penalty favors the symmetric representation.

For component f , reshape the corresponding rows of Θ_{uu} and Θ_{cc} into square matrices $Q_{uu}^{(f)}$ and $Q_{cc}^{(f)}$. A quadratic term can then be written as $x^\top Qx$. For any real square matrix,

$$x^\top Qx = x^\top \text{sym}(Q)x, \quad \text{sym}(Q) = \frac{Q + Q^\top}{2}. \quad (1.16)$$

The skew-symmetric part contributes nothing to the prediction. It can nevertheless increase the coefficient norm because

$$\|Q\|_F^2 = \|\text{sym}(Q)\|_F^2 + \|\text{skew}(Q)\|_F^2. \quad (1.17)$$

Replacing each square slice by its symmetric part preserves every fitted value and cannot increase a positive ridge penalty. The projection is applied after each driver update. With a strictly positive driver ridge weight, the exact minimum-norm solution has no skew-symmetric part.

Symmetry also determines how a coefficient display is read. If $q_{ij} = q_{ji}$, the two displayed off-diagonal cells together contribute $2q_{ij}x_ix_j$. A displayed cell is therefore half the coefficient of the corresponding unordered cross term. For an illustrative matrix with $q_{12} = q_{21} = 0.3$, the polynomial contribution is $0.6x_1x_2$. Reading either cell as the entire interaction coefficient would halve its contribution.

Each component has $M_{op}(M_{op} + 1)/2 = 3$ distinct operating quadratic terms and $F(F + 1)/2 = 210$ distinct influent quadratic terms. Together with the baseline, linear terms, and operating-load interactions, the library has 276 distinct polynomial terms per component. These are structural dimensions of the model. They do not describe how many coefficients are statistically important or biologically meaningful.

1.4 Learned coupling among effluent components

Consider two hypothetical component equations in which each channel has a coupling coefficient 0.2 to the other channel. With driver values 4 and 2, their equations are

$$c_1 = 4 + 0.2c_2, \quad c_2 = 2 + 0.2c_1. \quad (1.18)$$

Substituting the second equation into the first gives $c_1 = 4 + 0.2(2 + 0.2c_1)$. Moving the remaining c_1 term to the left gives $0.96c_1 = 4.4$. Therefore $c_1 = 55/12$ and $c_2 = 35/12$. The solution is approximately $(4.5833, 2.9167)^\top$. These arithmetic values demonstrate a simultaneous regression relation. They do not describe a biochemical conversion.

Moving the coupled terms to the left makes the same calculation a matrix solve,

$$\Gamma = \begin{bmatrix} 0 & 0.2 \\ 0.2 & 0 \end{bmatrix}, \quad R = \begin{bmatrix} 1 & -0.2 \\ -0.2 & 1 \end{bmatrix}, \quad R \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}. \quad (1.19)$$

The determinant of R is $1 - 0.2^2 = 0.96$. Its inverse is

$$R^{-1} = \frac{1}{0.96} \begin{bmatrix} 1 & 0.2 \\ 0.2 & 1 \end{bmatrix}. \quad (1.20)$$

Multiplying this inverse by the driver gives the same two scalar solutions. The inverse also explains why a change in one driver can change both predicted components.

The regression includes a matrix $\Gamma \in \mathbb{R}^{F \times F}$ that describes fitted dependence among the effluent components. Define

$$R = I_F - \Gamma, \quad Rc = r(u, c_{in}). \quad (1.21)$$

Equivalently, the equation for component f is $c_f = r_f + \sum_{j \neq f} \Gamma_{fj} c_j$. This form separates the input-dependent driver from dependence among the predicted component channels. The diagonal of Γ is fixed to zero to avoid a redundant self-coupling coefficient.

The sign of Γ_{fj} describes reinforcement or compensation within this fitted simultaneous relation. It does not describe a one-way biochemical reaction from component j to component f . Activated sludge reactions involve several components at once, and the stoichiometric matrix already specifies those transformations. The coupling matrix is learned from state patterns. Its entries can reflect correlated transformations, loading patterns, and the allocation of effects between the driver and coupling blocks.

When R is nonsingular, the raw component prediction is $R^{-1}r$. The dependence on R^{-1} means that the sign of a driver coefficient alone does not determine the response of the raw component prediction. Coupling can redistribute or amplify that response. For interpretation, the driver, coupling matrix, and complete raw mapping are examined together.

The adopted coupling constraints require

$$\Gamma_{ff} = 0, \quad |\Gamma_{fj}| \leq \gamma_{abs} \quad (f \neq j), \quad \kappa_2(I_F - \Gamma) \leq \kappa_{max}. \quad (1.22)$$

The condition number is the ratio of the largest to smallest singular value of the coupled-system matrix. A nearly singular matrix can amplify small driver perturbations and numerical errors. Bounding individual entries does not by itself guarantee a well-conditioned matrix, so the condition check is separate from the box constraints.

The small symmetric matrix in Equation (1.19) illustrates this measure without a large singular-value calculation. In the direction $(1, 1)^\top$, multiplication by R scales the vector by 0.8. In the direction $(1, -1)^\top$, it scales the vector by 1.2. These orthogonal directions give singular values 0.8 and 1.2, so the condition number is $1.2/0.8 = 1.5$.

If the two illustrative couplings were 0.99, the same directions would have scales 0.01 and 1.99. The condition number would be 199. A driver error of magnitude ϵ in each component along the first direction would produce a state error of 100ϵ in each component. This hypothetical choice is outside the adopted coefficient bound and serves only to show the mechanism of amplification.

Even a small entry bound can permit singularity in a larger matrix. In a hypothetical four-component matrix with every off-diagonal entry equal to $1/3$, every row sum of Γ is one. Hence $\Gamma \mathbf{1} = \mathbf{1}$ and $(I - \Gamma)\mathbf{1} = 0$. Each entry is smaller than the adopted bound 0.368, but the coupled-system matrix is singular. This example explains why entry bounds and the condition-number test perform separate tasks.

Shrinking a candidate coupling matrix toward zero can provide a safe candidate because $I_F - 0 = I_F$. A zero coupling matrix gives a condition number of one. A prescribed $\kappa_{max} > 1$ therefore permits this fallback. The admissible coupling set contains a nonlinear condition-number requirement. It is not generally a convex set, even though the box-constrained estimation problem used to propose a coupling update is convex.

1.5 Fitting in prediction space

1.5.1 The coupled training objective

The squared error of one prediction is the square of its difference from the reference. For several components and observations, fitting adds these squared differences. A compact matrix norm represents this sum. For any matrix M with entries M_{ij} ,

$$\|M\|_F^2 = \sum_i \sum_j M_{ij}^2. \quad (1.23)$$

For example, a residual matrix $\begin{bmatrix} 1 & -2 \\ 3 & 0 \end{bmatrix}$ has squared norm $1^2 + (-2)^2 + 3^2 + 0^2 = 14$.

The norm is $\sqrt{14}$, while a least-squares objective uses 14. Squaring prevents positive and negative errors from canceling when they are added.

Let i identify an observation, f a component, k an invariant, and ℓ a feature. The entry $\hat{c}_{if}$ is the fitted concentration, while $\phi_{i\ell}$ is the corresponding feature entry. Before

grouping the five training contributions into norms, their scalar form is

$$\begin{aligned}
J = & \sum_{i=1}^N \sum_{f=1}^F (c_{out,if} - \hat{c}_{if})^2 \\
& + \lambda_{inv} \sum_{i=1}^N \sum_{k=1}^K \left[\sum_{f=1}^F A_{kf} (\hat{c}_{if} - c_{in,if}) \right]^2 \\
& + \lambda_{sys} \sum_{i=1}^N \sum_{f=1}^F \left[\hat{c}_{if} - \sum_{j=1}^F \Gamma_{fj} \hat{c}_{ij} - \sum_{\ell=1}^D B_{f\ell} \phi_{i\ell} \right]^2 \\
& + \lambda_B \sum_{f=1}^F \sum_{\ell=1}^D B_{f\ell}^2 + \lambda_{\Gamma} \sum_{f=1}^F \sum_{j=1}^F \Gamma_{fj}^2.
\end{aligned} \tag{1.24}$$

The second contribution first computes an inventory residual by adding component changes with their invariant weights. It then squares that residual. It does not square each component change before the invariant calculation. The third contribution compares the fitted component state, after coupling, with the driver generated by the inputs. The last two sums penalize coefficient magnitude rather than prediction residuals.

The dimensions of the matrices show how these sums are assembled. The entry in row i , column k of $(\hat{C} - C_{in})A^\top$ is the inventory residual in the second sum. The entry in row i , column f of $\hat{C}R^\top$ is $\hat{c}_{if} - \sum_j \Gamma_{fj} \hat{c}_{ij}$. The transpose appears because observations are stored as rows, while the single-observation equation $Rc = r$ uses column vectors.

For N training observations, $\Phi \in \mathbb{R}^{N \times D}$ contains feature vectors as rows. The matrices C_{in} and C_{out} contain influent and reference effluent states in the same row convention. An auxiliary fitted-state matrix $\hat{C} \in \mathbb{R}_+^{N \times F}$ separates the predicted training states from the regression coefficients. The training criterion is

$$\begin{aligned}
J(B, \Gamma, \hat{C}) = & \|C_{out} - \hat{C}\|_F^2 + \lambda_{inv} \|(\hat{C} - C_{in})A^\top\|_F^2 \\
& + \lambda_{sys} \|\hat{C}(I_F - \Gamma)^\top - \Phi B^\top\|_F^2 \\
& + \lambda_B \|B\|_F^2 + \lambda_{\Gamma} \|\Gamma\|_F^2.
\end{aligned} \tag{1.25}$$

The estimation problem minimizes this criterion subject to nonnegative fitted training states, zero coupling diagonal, coupling box bounds, and the conditioning safeguard. All penalty weights are nonnegative. The adopted ridge weights and coupled-system weight are strictly positive.

The first term measures component prediction error. The second term discourages disagreement with the modeled invariants. The third term encourages agreement between the fitted states and the coupled regression equation. The final two terms limit coefficient magnitude. A nonnegative fitted training state can therefore balance prediction error, invariant mismatch, and coupled-system mismatch. It need not satisfy either equality exactly.

An illustrative one-observation calculation makes the five contributions concrete. Take $c_{out} = (3, 1)^\top$, $c_{in} = (2, 2)^\top$, $\hat{c} = (2.5, 1.2)^\top$, and $A = [1 \ 1]$. Use the hypothetical coupling matrix in Equation (1.19). For a single baseline feature, take $B = (2, 1)^\top$, so the driver is $(2, 1)^\top$. The five unweighted contributions are

$$\begin{aligned}
\text{prediction error} &= (3 - 2.5)^2 + (1 - 1.2)^2 = 0.29, \\
\text{inventory error} &= (2.5 + 1.2 - 2 - 2)^2 = 0.09, \\
\text{system error} &= (2.5 - 0.2(1.2) - 2)^2 \\
&\quad + (1.2 - 0.2(2.5) - 1)^2 = 0.1576, \\
\text{driver squared sum} &= 2^2 + 1^2 = 5, \\
\text{coupling squared sum} &= 0.2^2 + 0.2^2 = 0.08.
\end{aligned} \tag{1.26}$$

With illustrative weights $\lambda_{inv} = 2$, $\lambda_{sys} = 3$, $\lambda_B = 0.1$, and $\lambda_\Gamma = 0.5$, the objective is $0.29 + 2(0.09) + 3(0.1576) + 0.1(5) + 0.5(0.08) = 1.4828$. These weights are chosen only for arithmetic teaching. The adopted numerical fitting weights remain those in Table 1.1. The example shows why changing a weight changes the relative cost of a discrepancy rather than making that discrepancy an exact equality.

This distinction matters for a new input. Neither a nonnegative $\hat{C}$ nor a finite invariant penalty guarantees that $R^{-1}B\phi$ is nonnegative or conservative. The auxiliary state is a variable in the fitting problem, while the raw prediction is obtained by solving the learned coupled relation. A soft residual penalty encourages agreement but does not impose equality. Physics-informed residual training uses this general penalty principle, as developed by ?. Exact feasibility requires the separate deployment conditions below.

The objective is expressed in physical component coordinates. Components have different units and numerical scales, so the magnitudes of their residuals and coefficients reflect those conventions. The four penalty weights also depend on the fixed feature and invariant scaling. They cannot be transferred unchanged to arbitrary unit systems. Any rescaling must transform the corresponding features, coefficients, invariant operator, and reporting map consistently. ? discuss the influence of feature scaling on learning. For physical interpretation, unit consistency is as necessary as numerical conditioning.

1.5.2 Deterministic block updates

The term $\hat{C}\Gamma^\top$ makes the joint objective nonconvex. Holding two variable blocks fixed gives a simpler problem in the remaining block. The fitting procedure updates the driver, coupling matrix, and fitted states in that order within each cycle.

This distinction can be checked with the reduced residual $(1 - \gamma z)^2$, where γ is a coupling and z a fitted component. If z is fixed, this is a quadratic function of γ . If γ is fixed, it is a quadratic function of z . It is not jointly convex. At $(\gamma, z) = (0.2, 5)$ and $(0.1, 10)$, the residual is zero. At the midpoint $(0.15, 7.5)$, it is $(1 - 1.125)^2 = 1/64$.

A convex function could not exceed the average of the two zero endpoint values at their midpoint. This small example identifies the product responsible for the lack of a general joint-convexity guarantee.

For fixed Γ and $\hat{C}$, the driver update is a ridge-regularized linear least-squares problem. Define $T = \hat{C}R^\top$. For one output row, write its coefficient column as β and its target column from T as t . The part of the objective that depends on this row is

$$\lambda_{sys} \sum_{i=1}^N \left(t_i - \sum_{\ell=1}^D \phi_{i\ell} \beta_\ell \right)^2 + \lambda_B \sum_{\ell=1}^D \beta_\ell^2. \quad (1.27)$$

Differentiate with respect to one coefficient β_h . The derivative of the squared residual contributes its residual times the feature ϕ_{ih} , while the ridge term contributes $2\lambda_B \beta_h$. Setting the derivative to zero gives

$$\lambda_{sys} \sum_{i=1}^N \phi_{ih} \left(\sum_{\ell=1}^D \phi_{i\ell} \beta_\ell - t_i \right) + \lambda_B \beta_h = 0. \quad (1.28)$$

Writing this equation for every coefficient gives

$$\left(\lambda_{sys} \Phi^\top \Phi + \lambda_B I_D \right) \beta = \lambda_{sys} \Phi^\top t. \quad (1.29)$$

The matrix $\Phi^\top \Phi$ contains sums of products of feature columns. The vector $\Phi^\top t$ contains sums of feature times target. Adding the positive ridge weight to the diagonal prevents a zero-curvature direction when feature columns are duplicated.

For a hypothetical calculation with one coefficient and no intercept, let $\Phi = (1, 2)^\top$, $t = (2, 3)^\top$, and both weights equal one. The objective becomes

$$(2 - \beta)^2 + (3 - 2\beta)^2 + \beta^2 = 13 - 16\beta + 6\beta^2. \quad (1.30)$$

Its derivative is $-16 + 12\beta$, so the minimizer is $\beta = 4/3$. The matrix calculation gives $\Phi^\top \Phi = 1^2 + 2^2 = 5$, $\Phi^\top t = 1(2) + 2(3) = 8$, and $(5 + 1)\beta = 8$. Both calculations agree. Without the penalty, the coefficient would be $8/5$. The reduction illustrates ridge shrinkage for one coefficient. The adopted driver still includes its baseline and all stated features.

Assembling the normal equations for all output rows gives the exact solution

$$B^\top = \left(\lambda_{sys} \Phi^\top \Phi + \lambda_B I_D \right)^{-1} \lambda_{sys} \Phi^\top \hat{C} R^\top. \quad (1.31)$$

The positive driver ridge weight makes the matrix in this expression positive definite, even when ordered quadratic columns are duplicated. In computation, the corresponding linear system is solved. An explicit matrix inverse is unnecessary. The square quadratic slices are then symmetrized without changing their predictions.

For fixed B and $\hat{C}$, define $E = \hat{C} - \Phi B^\top$.

The coupling calculation can be read one output row at a time. For row f , the residual at observation i is $E_{if} - \sum_{j \neq f} \hat{c}_{ij} \Gamma_{fj}$. Its minimization uses the other fitted

component columns as predictors of E_{if} , with a ridge penalty and box bounds. In a hypothetical row with one permitted coupling γ , two predictor values 1 and 2, and residual targets 0.4 and 0.8, equal unit weights give

$$(0.4 - \gamma)^2 + (0.8 - 2\gamma)^2 + \gamma^2 = 0.8 - 4\gamma + 6\gamma^2. \quad (1.32)$$

The derivative $-4 + 12\gamma$ vanishes at $\gamma = 1/3$. This value lies below the adopted absolute entry bound 0.368. If the illustrative box bound were instead 0.25, the convex scalar problem would attain its minimum at 0.25. The conditioning test would still be applied to the complete coupling matrix. A row-wise scalar minimizer cannot certify the conditioning of that matrix.

Collecting these scalar residuals across all output rows gives the coupling proposal

$$\min_{\Gamma} \lambda_{sys} \|E - \hat{C}\Gamma^\top\|_F^2 + \lambda_\Gamma \|\Gamma\|_F^2 \quad (1.33)$$

subject to its zero diagonal and box bounds. This is a convex quadratic program. Its candidate is subsequently screened using Equation (1.22). A candidate that fails the condition check is shrunk toward zero or replaced by an admissible fallback. Screening preserves numerical admissibility, but it need not preserve the minimum of the unrestricted convex subproblem.

For fixed B and Γ , the fitted-state update separates over the training rows. For sample i , write $\hat{c}_i$ as a column vector.

Expand the three residual sums for one observation before collecting their coefficients. The data term contributes $\hat{c}^\top \hat{c} - 2c_{out}^\top \hat{c}$. The invariant term contributes $\lambda_{inv} \hat{c}^\top A^\top A \hat{c} - 2\lambda_{inv} c_{in}^\top A^\top A \hat{c}$. The system term contributes $\lambda_{sys} \hat{c}^\top R^\top R \hat{c} - 2\lambda_{sys} (B\phi)^\top R \hat{c}$. Terms independent of $\hat{c}$ do not affect its minimizer. Collecting its quadratic terms gives H_C , and collecting its linear terms gives h_i .

Write the expanded quadratic as $\hat{c}_i^\top H_C \hat{c}_i - 2h_i^\top \hat{c}_i$. The entry $(H_C)_{fg}$ multiplies the ordered product $\hat{c}_{if} \hat{c}_{ig}$, and $(h_i)_f$ supplies its linear term. For distinct components, the two mirrored entries contribute together, following the quadratic symmetry convention. Let δ_{fg} equal one when $f = g$ and zero otherwise. Reading the coefficients entry by entry gives

$$\begin{aligned} (H_C)_{fg} &= \delta_{fg} + \lambda_{inv} \sum_{k=1}^K A_{kf} A_{kg} + \lambda_{sys} \sum_{j=1}^F R_{jf} R_{jg}, \\ (h_i)_f &= c_{out,if} + \lambda_{inv} \sum_{k=1}^K A_{kf} \sum_{g=1}^F A_{kg} c_{in,ig} \\ &\quad + \lambda_{sys} \sum_{j=1}^F R_{jf} \sum_{\ell=1}^D B_{j\ell} \phi_{i\ell}. \end{aligned} \quad (1.34)$$

For example, the invariant contribution to $(H_C)_{12}$ adds the product of the first and second component weights over the invariant rows. The system contribution adds the product of columns 1 and 2 of R . These products explain why correcting one fitted component can change the preferred value of another.

Completing the square explains what the nonnegative solve is adjusting. With $c_0 = H_C^{-1}h_i$,

$$\hat{c}^\top H_C \hat{c} - 2h_i^\top \hat{c} = (\hat{c} - c_0)^\top H_C (\hat{c} - c_0) - h_i^\top H_C^{-1} h_i. \quad (1.35)$$

The unrestricted minimizer is c_0 . When it contains a negative entry, the constrained problem seeks the closest nonnegative state in the quadratic geometry defined by H_C . Off-diagonal entries tilt that geometry, so ordinary coordinate clipping need not give the closest state.

For example, take hypothetical $H_C = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$ and $h_i = (-1, 2)^\top$. These coefficients can arise from unit data, invariant, and system weights with $A = [1 \ 1]$ and $R = I$. A signed driver can make the first entry of h_i negative. The unrestricted solution is $c_0 = (-5/8, 7/8)^\top$. Clipping gives $(0, 7/8)^\top$. Instead, set the constrained first component to zero and minimize the remaining expression $3c_2^2 - 4c_2$. Its derivative $6c_2 - 4$ gives $c_2 = 2/3$.

At $(0, 2/3)^\top$, the derivative in the first coordinate is $2(c_2 + 1) = 10/3$, which is positive. Increasing that zero component would increase the objective, so the boundary value is appropriate. The objective is $-4/3$ at this state and $-77/64$ at the clipped state. The constrained solution has the smaller objective. The negative objective values are possible because the constants removed during expansion are absent. The complete residual-based training criterion remains nonnegative.

The general coefficient arrays derived above are assembled in the compact form

$$\begin{aligned} H_C &= I_F + \lambda_{inv} A^\top A + \lambda_{sys} R^\top R, \\ h_i &= c_{out,i} + \lambda_{inv} A^\top A c_{in,i} + \lambda_{sys} R^\top B \phi_i. \end{aligned} \quad (1.36)$$

Each row minimizes $\hat{c}_i^\top H_C \hat{c}_i - 2h_i^\top \hat{c}_i$ over $\hat{c}_i \geq \mathbf{0}$. The identity term makes H_C positive definite, so the row minimizer is unique. Clipping the unconstrained solution componentwise generally does not solve this problem because the off-diagonal terms in H_C connect component adjustments.

The quadratic-program method of ? provides an algorithmic foundation for the constrained coupling and fitted-state blocks. These blocks use their quadratic structure directly. A generic stochastic first-order method could also optimize a related criterion, but it would require separate mechanisms to control nonnegativity, coupling bounds, and conditioning. The block formulation keeps each requirement attached to an explicit subproblem.

1.5.3 Descent, stopping, and numerical safeguards

Exact minimization over a block cannot increase the objective when its admissible set contains the previous block value. Since $J \geq 0$, an objective sequence that is monotone non-increasing converges in value. This argument does not establish a global minimum

of the joint problem. The convergence analysis of ? specifies additional conditions needed to relate blockwise minimization to stationary solutions.

The conditioning screen requires particular care. Shrinking a coupling proposal can increase the training criterion relative to the previous admissible coupling matrix. The procedure therefore evaluates the objective after screening and retains the previous admissible matrix when a screened candidate cannot provide an acceptable non-increasing update. Solver tolerances also require an explicit allowance for numerical error. A claim of monotone descent applies to accepted objective-decreasing updates. It does not follow from box-constrained minimization alone.

Each restart begins with an admissible coupling matrix and nonnegative fitted states. A ridge solve supplies the initial driver. Full cycles continue until an iteration limit is reached or the slope of the trailing objective history is sufficiently small. A running minimum records the best admissible state reached. When several restarts are used, the admissible state with the smallest recorded objective is retained. A nearly flat objective indicates limited further progress under that stopping rule. It does not certify parameter uniqueness or global optimality.

Figure 1.1 separates the three fitting updates from the checks that make an update acceptable. The scalar calculations in Equations (1.30) and (1.32) illustrate the first and third computational boxes in the left column. The nonnegative-state example explains the return path in the right column.

The cycle returns to the driver because each block changes the conditions faced by the others. The conditioning screen acts on the coupling proposal before the fitted-state update. The full objective is then recorded after the cycle. The restart decision concerns the prescribed search procedure, while the final selection compares admissible objective values. These steps explain why a sequence of convex block solves does not establish a jointly global optimum.

Table 1.1: Specified fitting settings for the structured regression.

Setting	Value
Baseline feature	Included
Invariant penalty λ_{inv}	8.68×10^{-3}
Coupled-system penalty λ_{sys}	1.27×10^{-3}
Driver ridge penalty λ_B	6.09×10^{-5}
Coupling ridge penalty λ_Γ	6.55×10^{-4}
Off-diagonal bound γ_{abs}	0.368
Maximum full fitting cycles	60
Restarts	1
Trailing objective window	12 cycles
Objective slope tolerance	2.85×10^{-7}

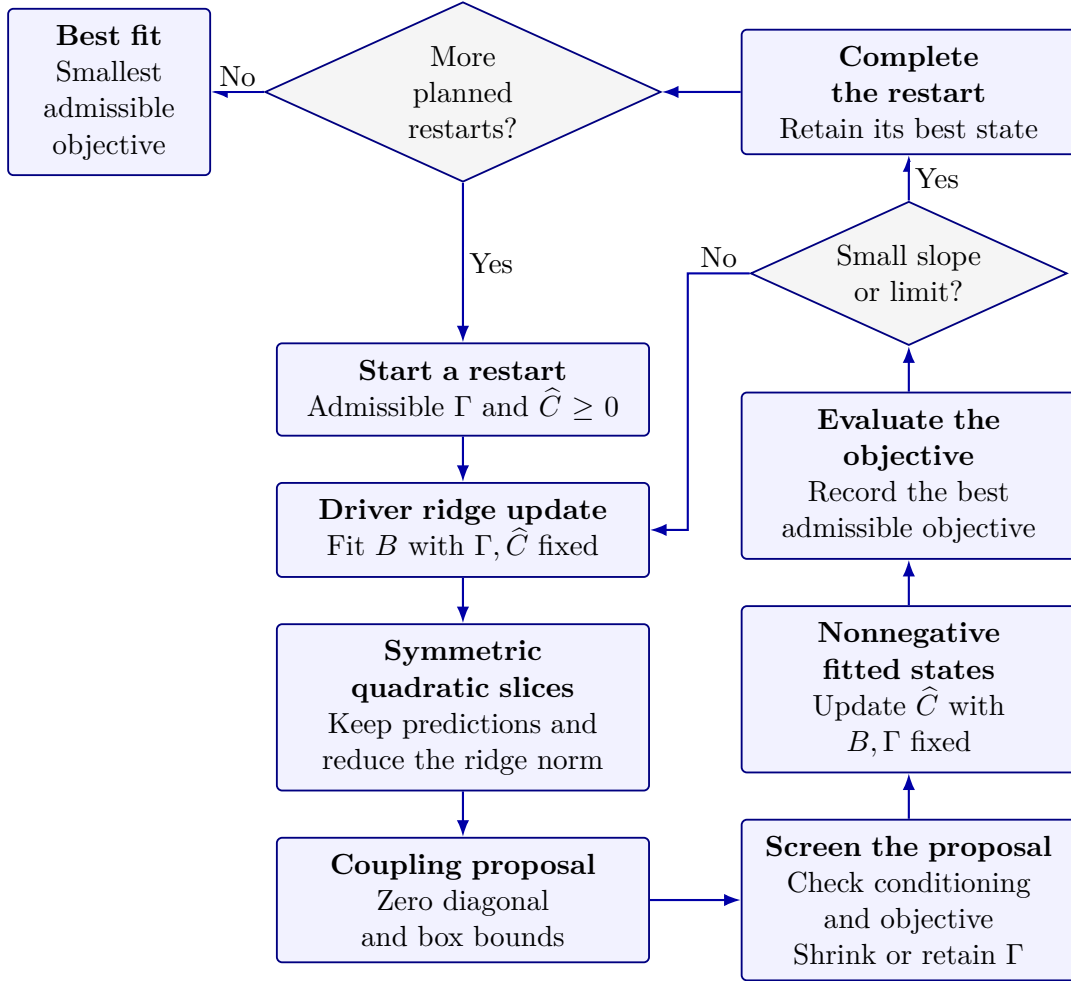


Figure 1.1: Conceptual fitting cycle with explicit symmetry, conditioning, descent, and nonnegativity checks. The prescribed restart count can be one.

Table 1.1 specifies numerical fitting choices. The condition threshold κ_{max} remains an additional prescribed numerical safeguard and is checked independently of the coefficient bound. Hyperparameter assessment uses validation observations within the training pool. Held-out test observations do not determine penalty weights or stopping choices. Fitting settings remain fixed within a sample-size comparison so that changes in the learning curve reflect changing sample availability under a common model specification.

1.6 Checked deployment

1.6.1 The raw coupled state

For a new pair $(u_*, c_{in,*})$, the fitted parameters give

$$\hat{R} = I_F - \hat{\Gamma}, \quad d_* = \hat{B}\phi(u_*, c_{in,*}), \quad \hat{R}c_{raw} = d_*. \quad (1.37)$$

The notation $c_{raw} = \hat{R}^{-1}d_*$ describes the mapping mathematically. A linear-system solve evaluates it numerically. The accepted conditioning bound limits the sensitivity of that solve. The raw state is returned only when it satisfies both the invariant and nonnegativity checks.

For a vector, the Euclidean norm is the square root of the sum of squared entries. Thus an illustrative invariant-residual vector $(3, 4)^\top$ has norm $\sqrt{3^2 + 4^2} = 5$. The absolute norm adds the absolute entries instead. For an illustrative concentration vector $(-0.3, 0.1, -0.4)^\top$, retaining only negative entries gives $(-0.3, 0, -0.4)^\top$, whose absolute norm is 0.7. The first calculation measures the size of a conservation discrepancy. The second measures total negative concentration in the stated numerical coordinates.

For any candidate c , define the diagnostic residuals

$$v_{mc}(c) = \|A(c - c_{in,*})\|_2, \quad v_{nn}(c) = \|\min(c, \mathbf{0})\|_1. \quad (1.38)$$

The minimum is taken componentwise. The adopted diagnostic tolerance is 10^{-10} . Algebraic equality and nonnegativity define the mathematical requirement. Residuals below the numerical tolerance define its computational assessment. Invariant residual magnitudes are meaningful only under the stated scaling of A .

1.6.2 Affine restoration of the invariants

The nearest conservative state can first be derived for the hypothetical two-component inventory. Write the raw prediction as $(p_1, p_2)^\top$ and the required total as T . The two unknown corrected concentrations minimize $(c_1 - p_1)^2 + (c_2 - p_2)^2$ while satisfying $c_1 + c_2 = T$. Introduce a multiplier η for that equality and write

$$\mathcal{L} = \frac{1}{2}(c_1 - p_1)^2 + \frac{1}{2}(c_2 - p_2)^2 + \eta(c_1 + c_2 - T). \quad (1.39)$$

The factor one-half simplifies differentiation without changing the minimizer. Differentiating with respect to the two concentrations and the multiplier gives

$$c_1 - p_1 + \eta = 0, \quad c_2 - p_2 + \eta = 0, \quad c_1 + c_2 = T. \quad (1.40)$$

The first two equations give $c_1 = p_1 - \eta$ and $c_2 = p_2 - \eta$. Substituting them into the inventory equality gives $p_1 + p_2 - 2\eta = T$. Thus

$$\begin{aligned} \eta &= \frac{p_1 + p_2 - T}{2}, \\ c_1 &= p_1 - \frac{p_1 + p_2 - T}{2}, \\ c_2 &= p_2 - \frac{p_1 + p_2 - T}{2}. \end{aligned} \quad (1.41)$$

The unweighted distance shares the inventory discrepancy equally between these two coordinates. With $p_1 = 12$, $p_2 = 1$, and $T = 10$, the discrepancy is 3 and the multiplier is 1.5. The projected state is $(10.5, -0.5)^\top$. The equality is satisfied even though the second component is negative. This example identifies the inequality that is absent from an equality-only projection.

When the raw state fails its checks, the first correction solves

$$c_{aff} = \arg \min_{c \in \mathbb{R}^F} \|c - c_{raw}\|_2^2 \quad \text{subject to} \quad Ac = Ac_{in,*}. \quad (1.42)$$

Introduce a Lagrange multiplier $\eta \in \mathbb{R}^K$. Stationarity gives $c - c_{raw} + A^\top \eta = 0$. Substitution into the equality gives $AA^\top \eta = A(c_{raw} - c_{in,*})$. The full row rank of A therefore gives the unique solution

$$c_{aff} = c_{raw} - A^\top (AA^\top)^{-1} A(c_{raw} - c_{in,*}). \quad (1.43)$$

The fixed matrix $A^\top (AA^\top)^{-1} A$ can be prepared once. The affine correction is then a matrix-vector operation for each new prediction.

Each matrix step follows the same scalar derivation. The multiplier column contains one multiplier per independent inventory. The vector $A^\top \eta$ converts these inventory corrections into component adjustments. Solving $AA^\top \eta = A(c_{raw} - c_{in,*})$ determines the adjustments needed to remove the inventory residual. For the two-component illustration,

$$AA^\top = 2, \quad A^\top (AA^\top)^{-1} A = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}. \quad (1.44)$$

Taking the illustrative influent as $(6, 4)^\top$, the raw-minus-influent difference is $(6, -3)^\top$. Multiplying it by this projection matrix gives $(1.5, 1.5)^\top$, which is subtracted from the raw state. This recovers the scalar calculation without guessing how much either component should change.

This correction is an orthogonal projection onto the invariant-consistent affine set in the chosen numerical component coordinates. It removes the part of the prediction

error normal to that set. It retains the part tangent to the set, which can still be large. The correction therefore enforces bookkeeping without establishing predictive accuracy. Its distance also depends on the component unit convention. ? develop related conservation-restoring corrections and explain why species-dependent weights matter when concentration scales differ.

The affine state is checked for both residual conservation and nonnegativity before deployment. The equality follows analytically from Equation (1.43), but checking its residual catches numerical or operator inconsistency. A conservative affine state can still contain a negative concentration. That possibility requires a correction with inequality constraints.

1.6.3 Joint conservation and nonnegativity

The absolute-distance correction can also be written before introducing compact vector inequalities. For the conservative but negative state $(10.5, -0.5)^\top$, require $c_1 + c_2 = 10$ and $c_1, c_2 \geq 0$. Introduce nonnegative adjustment bounds ρ_1, ρ_2 . The constraints

$$\begin{aligned} \rho_1 &\geq c_1 - 10.5, & \rho_1 &\geq 10.5 - c_1, \\ \rho_2 &\geq c_2 + 0.5, & \rho_2 &\geq -0.5 - c_2 \end{aligned} \tag{1.45}$$

make ρ_1 at least the absolute first adjustment and ρ_2 at least the absolute second adjustment. Minimizing $w_1\rho_1 + w_2\rho_2$ with positive weights makes each bound equal to the corresponding absolute adjustment. No absolute-value function remains in the optimization constraints.

Set $c_2 = t$ to impose the inventory equality, so $c_1 = 10 - t$ and $0 \leq t \leq 10$. Both absolute adjustments then equal $t + 0.5$. The objective is $(w_1 + w_2)(t + 0.5)$, which increases with t . Its minimum is at $t = 0$, giving $(10, 0)^\top$. For equal unit weights, its cost is 1. At $t = 1$, the state would be $(9, 1)^\top$, with cost 3. This direct comparison explains the optimal correction in the reduced problem.

When the affine state fails the nonnegativity check, the final correction solves

$$\begin{aligned} \hat{c} &= \arg \min_{c, \rho \in \mathbb{R}^F} w_c^\top \rho \\ \text{subject to} \quad &Ac = Ac_{in,*}, \\ &c \geq \mathbf{0}, \\ &-\rho \leq c - c_{aff} \leq \rho, \\ &\rho \geq \mathbf{0}. \end{aligned} \tag{1.46}$$

For strictly positive fixed weights w_c , minimization makes $\rho_f = |c_f - c_{aff,f}|$ at an optimum. The objective is therefore the weighted absolute adjustment $\sum_f w_{c,f} |c_f - c_{aff,f}|$. The reference is the affine state. The program does not minimize distance to the original raw state or re-estimate the regression coefficients.

The program is linear because the absolute values are represented by auxiliary variables and linear inequalities. The revised-simplex method described by ? provides an algorithmic foundation for solving this type of correction. The mathematical feasible set is nonempty whenever the supplied influent state is nonnegative under the adopted boundary relation. The positive weights make the adjustment objective bounded below and discourage unconstrained movement of any component. A minimizer exists, although several minimizing states can exist when the absolute-distance objective has a flat face.

Large weights make changes to a component more costly. Small weights permit larger changes when those changes help restore feasibility. Weights are fixed using the intended component scaling and scientific priorities. They are not chosen after seeing the reference target for a deployed observation. A sensitivity analysis of the weights can examine how much the returned state depends on that modeling choice. Equal numerical weights also embody a choice because component units and typical magnitudes differ.

Three hypothetical components show how weights can change an admissible allocation. Let the inventory row be $[1 \ 1 \ 1]$ and the conservative affine reference be $(-1, 4, 7)^\top$, whose total is 10. Restoring its first component to zero requires removal of one unit from another component. Two feasible candidates are $(0, 3, 7)^\top$ and $(0, 4, 6)^\top$. For weights $(1, 10, 1)^\top$, their costs are respectively $1(1)+10(1) = 11$ and $1(1)+1(1) = 2$. The second component is more costly to change, so removing material from the third is preferable. With weights $(1, 1, 10)^\top$, the two costs become 2 and 11. The preferred allocation reverses. Both allocations preserve the same inventory, and neither preference is supplied by conservation alone.

? show why a conservation correction needs simultaneous positivity control in reaction-kinetics surrogates. Their scaled-space treatment protects low-concentration species during correction. Equation (1.46) addresses the same joint requirement through a weighted absolute-distance problem for activated sludge components. It imposes the adopted linear invariants and nonnegativity on the returned state.

1.6.4 Illustrative correction and the failure of clipping

Consider an illustrative two-component system with $A = [1 \ 1]$, a total influent inventory of 10, and a raw prediction $c_{raw} = (12, 1)^\top$. The raw total is 13. The affine formula subtracts 1.5 from each component and gives $c_{aff} = (10.5, -0.5)^\top$. The total is now 10, but the second component is negative.

Clipping that negative value to zero produces $(10.5, 0)^\top$, which violates the total inventory. With equal correction weights, Equation (1.46) instead returns $(10, 0)^\top$. Both constraints hold. These values are a hypothetical illustration of the correction geometry. They are not activated sludge predictions or experimental findings.

The deployment order uses this geometry directly. A feasible raw state needs no

adjustment. An infeasible raw state is first projected onto the conservation set. A feasible affine state can then be returned. The linear program is required only when the affine state still fails nonnegativity. Every returned candidate is checked under the declared numerical tolerances. An unsuccessful numerical solve does not authorize an unchecked prediction.

Figure 1.2 makes the branch conditions explicit. Both physical checks appear at the raw and affine stages. The last correction branch also requires successful optimization and an independent admissibility check.

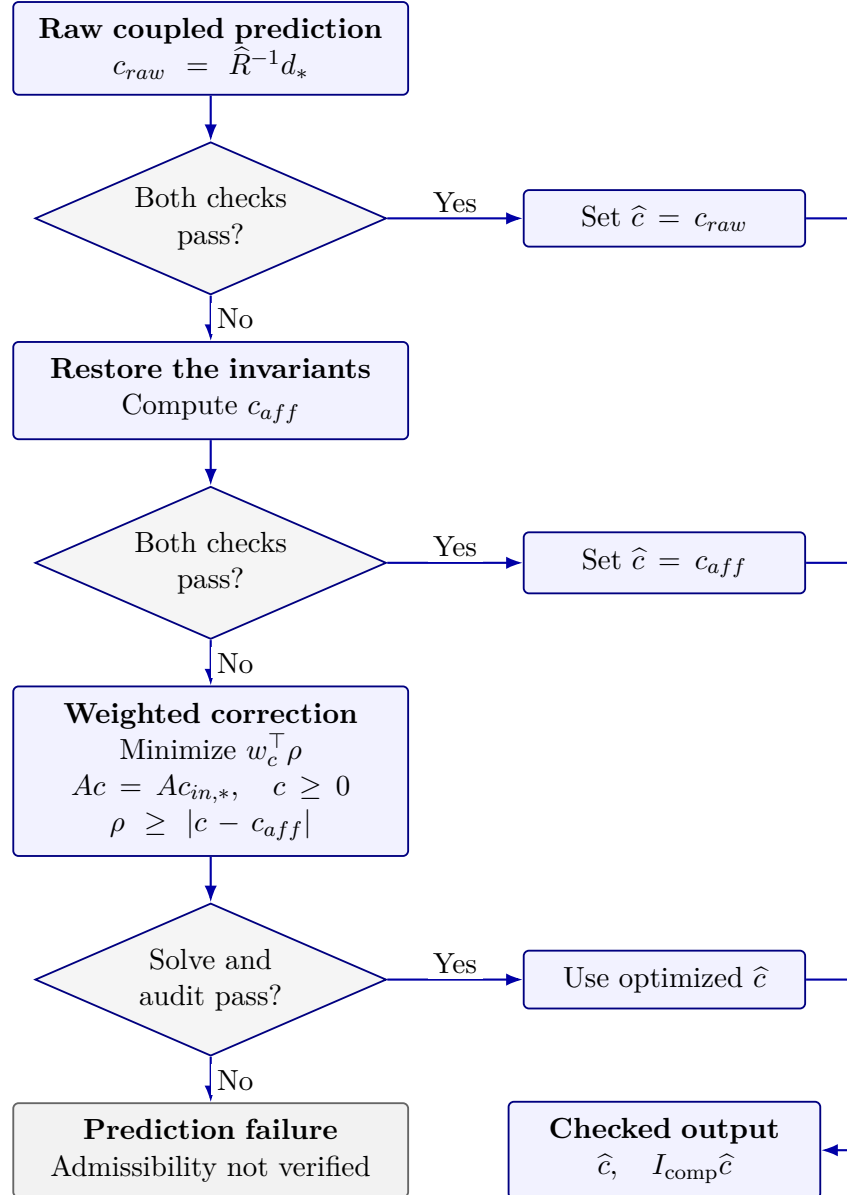


Figure 1.2: Checked deployment of one predicted component state. Accepted branches merge before the common reporting map is applied. A failed final audit produces a reported prediction failure.

The hypothetical state $(12, 1)^\top$ follows the two No branches because its total is

incorrect and its affine correction contains a negative component. The weighted correction supplies $(10, 0)^\top$, which reaches the return branch only after the stated checks pass. A state that satisfies both conditions at an earlier branch reaches the same reporting step. The different paths therefore change the correction effort while retaining the same acceptance requirements.

The component state returned by this rule is the deployed output. Its reported quantities are

$$\hat{y}_{\text{comp}} = I_{\text{comp}} \hat{c}. \quad (1.47)$$

The deployment constraint is responsible for admissibility. The soft training penalties can reduce the need for correction, but that possibility is an evaluation question rather than a mathematical guarantee.

1.7 Interpreting the complete prediction map

1.7.1 Driver coefficients and back projection

The transformation from driver coefficients to output coefficients can be computed with small matrices. Use the hypothetical coupling matrix in Equation (1.19), a reduced feature vector $(1, a)^\top$, and driver coefficients

$$B = \begin{bmatrix} 4 & 1 \\ 2 & 0 \end{bmatrix}, \quad r = \begin{bmatrix} 4 + a \\ 2 \end{bmatrix}. \quad (1.48)$$

The first coefficient column describes the baseline, and the second describes dependence on the hypothetical scalar input a . Multiplication through the coupling inverse gives

$$\begin{aligned} R^{-1}B &= \frac{1}{0.96} \begin{bmatrix} 1 & 0.2 \\ 0.2 & 1 \end{bmatrix} \begin{bmatrix} 4 & 1 \\ 2 & 0 \end{bmatrix} \\ &= \frac{1}{0.96} \begin{bmatrix} 4.4 & 1 \\ 2.8 & 0.2 \end{bmatrix} = \begin{bmatrix} 55/12 & 25/24 \\ 35/12 & 5/24 \end{bmatrix}. \end{aligned} \quad (1.49)$$

The second component now depends on a , even though its driver coefficient for a was zero. This dependence is introduced by the simultaneous coupling solve.

For a hypothetical reported quantity $y = c_1 + 2c_2$, its reporting row is $[1 \ 2]$. Its transformed coefficient row is

$$[1 \ 2] R^{-1}B = [125/12 \ 35/24]. \quad (1.50)$$

At $a = 2$, the driver is $(6, 2)^\top$, the raw component state is $(20/3, 10/3)^\top$, and the reported quantity is $40/3$. The transformed row gives the same value through $125/12 + (35/24)2 = 40/3$. This reporting row is chosen for teaching and is not an adopted COD or nutrient composition row.

The rows of $\hat{B}$ refer to component drivers. They cannot be labeled directly as COD, TN, TP, or TSS coefficients. The raw component coefficient matrix is

$$B_{\text{raw}} = \hat{R}^{-1} \hat{B}, \quad c_{\text{raw}} = B_{\text{raw}} \phi. \quad (1.51)$$

The corresponding map in reported-composite space is

$$B_{\text{raw,comp}} = I_{\text{comp}} \hat{R}^{-1} \hat{B}, \quad y_{\text{raw,comp}} = B_{\text{raw,comp}} \phi. \quad (1.52)$$

This transformation carries the driver coefficients through the learned coupling and then through the reporting operator. Inspection of a COD quadratic block uses its row in $B_{\text{raw,comp}}$, rather than an arbitrary row of $\hat{B}$.

Figure 1.3 shows the complete raw prediction chain using the named feature groups. It keeps the component driver, simultaneous coupling solve, and reporting operation visible as separate mathematical steps.

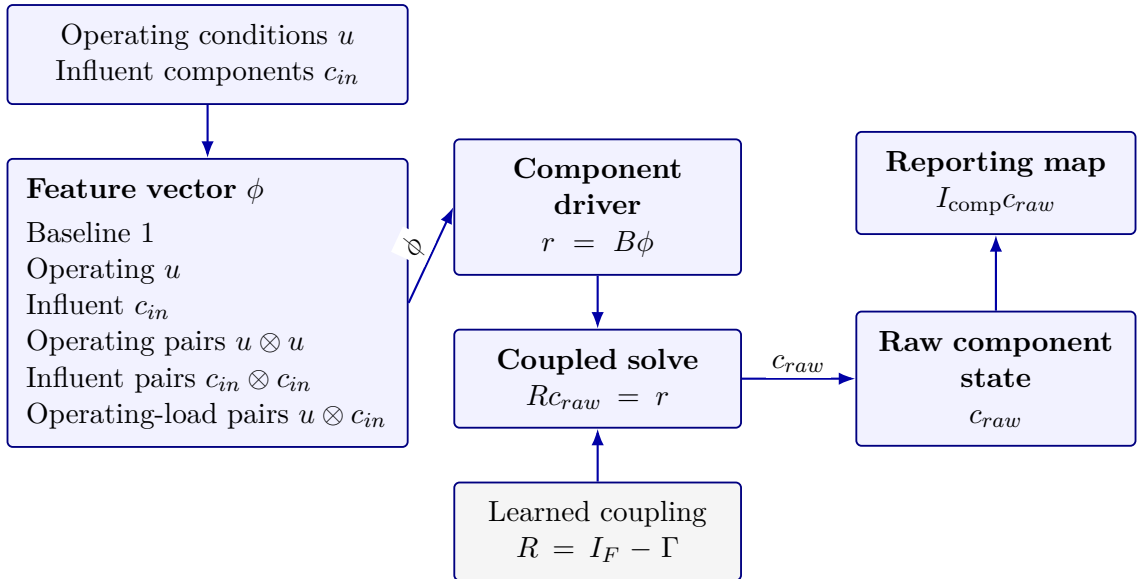


Figure 1.3: Named feature blocks and the transformation from driver to raw component and reported-composite predictions. Composing the three operations gives the raw reported coefficient map $I_{\text{comp}}R^{-1}B$.

In the two-feature example, the second driver has no direct coefficient for the scalar input, but the coupling path gives its raw component a nonzero input coefficient. The reporting path combines both component coefficients. Reading the paths in this order explains the back projection in Equations (1.49) and (1.50). Checked deployment acts on the raw component state before the final reported quantities are calculated.

The named blocks can be displayed as coefficient maps. Operating terms are grouped separately from influent terms. Symmetric quadratic maps display both ordered cells while reporting unique cross terms consistently. Operating-load maps show

which operating variables interact with which influent components. The coupling matrix is displayed separately because it has a different interpretation from the input-driver terms.

A coefficient-display threshold is used only to simplify reporting after fitting. For each reported-composite raw-head map, the threshold is 10^{-4} times its largest absolute coefficient. For the coupling matrix, the threshold is 10^{-4} times its largest absolute off-diagonal entry. The threshold does not modify the fitted mapping, select hyperparameters, or impose sparsity during training. Symmetric quadratic summaries count unique upper-triangular terms. A small coefficient can still have a large contribution when multiplied by a large feature. Interpretation therefore examines coefficient magnitude, input scale, and the contribution of the corresponding term over the declared domain.

1.7.2 Analytical jacobians of the raw head

Differentiation of a scalar polynomial shows why a coefficient and a marginal response differ. For the illustrative driver in Equation (1.10),

$$\begin{aligned}\frac{\partial r_1}{\partial u_1} &= 2 + 0.2u_1 + 0.2u_2, \\ \frac{\partial r_1}{\partial u_2} &= -1 + 0.2u_1 + 0.25c_1, \\ \frac{\partial r_1}{\partial c_1} &= 0.5 + 0.25u_2, \quad \frac{\partial r_1}{\partial c_2} = -0.1c_2.\end{aligned}\tag{1.53}$$

At $u = (2, 3)^\top$ and $c = (4, 5)^\top$, these derivatives are 3, 0.4, 1.25, and -0.5 . In particular, the first derivative is 3 although the linear coefficient of u_1 is 2. The operating quadratic term and interaction with u_2 supply the remaining contribution.

The derivative of the second illustrative driver $r_2 = -u_1 + u_2 + 0.2c_2$ with respect to u_1 is -1 . Differentiate the coupled equations with the coupling held fixed. Their derivative equations are $\partial c_1/\partial u_1 - 0.2\partial c_2/\partial u_1 = 3$ and $-0.2\partial c_1/\partial u_1 + \partial c_2/\partial u_1 = -1$. Thus the same inverse used for prediction gives

$$\begin{bmatrix} \partial c_1/\partial u_1 \\ \partial c_2/\partial u_1 \end{bmatrix} = \frac{1}{0.96} \begin{bmatrix} 1 & 0.2 \\ 0.2 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 35/12 \\ -5/12 \end{bmatrix}.\tag{1.54}$$

For the hypothetical reporting row $[1 \ 2]$, the reported derivative is $35/12 + 2(-5/12) = 25/12$. Differentiating the driver, solving through the coupling, and applying the reporting weights are three separate operations.

Putting several derivatives in one matrix gives a Jacobian. Its rows identify the output being differentiated, and its columns identify the input being varied. At the illustrative inputs, the operating-driver Jacobian and its coupled transformation are

$$J_{r,u} = \begin{bmatrix} 3 & 0.4 \\ -1 & 1 \end{bmatrix}, \quad R^{-1}J_{r,u} = \begin{bmatrix} 35/12 & 5/8 \\ -5/12 & 9/8 \end{bmatrix}.\tag{1.55}$$

The first column is the derivative calculation above. The second follows by solving the same coupled system for the derivative column $(0.4, 1)^\top$. A Jacobian therefore collects several scalar sensitivity calculations rather than introducing a different kind of response.

A coefficient describes a term in the polynomial. A Jacobian describes the change in the whole mapping at an operating point. For component f , let $H_{uc}^{(f)} \in \mathbb{R}^{M_{op} \times F}$ reshape its interaction coefficients so that the cross term is $u^\top H_{uc}^{(f)} c_{in}$. The driver can then be expressed as

$$r_f = b_f + (W_u)_{f,:} u + (W_{in})_{f,:} c_{in} + u^\top Q_{uu}^{(f)} u + c_{in}^\top Q_{cc}^{(f)} c_{in} + u^\top H_{uc}^{(f)} c_{in}. \quad (1.56)$$

With symmetric square slices, its gradients are

$$\begin{aligned} \nabla_u r_f &= (W_u)_{f,:}^\top + 2Q_{uu}^{(f)} u + H_{uc}^{(f)} c_{in}, \\ \nabla_{c_{in}} r_f &= (W_{in})_{f,:}^\top + 2Q_{cc}^{(f)} c_{in} + (H_{uc}^{(f)})^\top u. \end{aligned} \quad (1.57)$$

The quadratic gradients in Equation (1.57) follow by differentiating each scalar product. For $x^\top Q x = \sum_i \sum_j q_{ij} x_i x_j$, differentiation with respect to x_h collects the terms with $i = h$ and those with $j = h$. Their sum is $\sum_j (q_{hj} + q_{jh}) x_j$. It becomes the h th entry of $2Qx$ when Q is symmetric. For $u^\top H c = \sum_i \sum_j h_{ij} u_i c_j$, differentiation with respect to u_i gives $\sum_j h_{ij} c_j$, while differentiation with respect to c_j gives $\sum_i h_{ij} u_i$. These are the entries of Hc and $H^\top u$.

Stacking the transposed gradients gives $J_{r,u} \in \mathbb{R}^{F \times M_{op}}$ and $J_{r,c} \in \mathbb{R}^{F \times F}$. The raw-state and raw-composite Jacobians follow through the same back projection as the coefficients,

$$\begin{aligned} J_{raw,u} &= \hat{R}^{-1} J_{r,u}, & J_{raw,c} &= \hat{R}^{-1} J_{r,c}, \\ J_{raw,comp,u} &= I_{comp} J_{raw,u}, & J_{raw,comp,c} &= I_{comp} J_{raw,c}. \end{aligned} \quad (1.58)$$

These derivatives describe the complete fitted raw head at the specified inputs. They include curvature and interactions. They do not include the physical correction applied during deployment.

An illustrative aeration sensitivity can be inspected at low and high ammonium loading while holding the remaining inputs fixed. A difference in the two derivatives identifies a loading-dependent association within the fitted model. Oxygen-transfer limitations and heterotrophic competition provide process context for assessing such a pattern, as discussed by ?. Agreement with that context is a plausibility check. It is not evidence that changing aeration in a plant will cause exactly the fitted response.

1.7.3 Sensitivity after the affine correction

Return to the two-feature example in Equation (1.49). Its raw derivative with respect to a is $(25/24, 5/24)^\top$, whose total is $5/4$. This raw response changes the total component

inventory. At fixed influent inventory, an affine correction must remove that total change. For the two-component invariant,

$$P_A = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad (1.59)$$

$$\frac{\partial c_{aff}}{\partial a} = P_A \begin{bmatrix} 25/24 \\ 5/24 \end{bmatrix} = \begin{bmatrix} 5/12 \\ -5/12 \end{bmatrix}.$$

The two entries now sum to zero. For the reporting row $[1 \ 2]$, the derivative is $5/12 + 2(-5/12) = -5/12$. The corresponding raw derivative was $35/24$. The correction changes both its magnitude and sign because it enforces a different material-accounting condition on the complete returned state.

The influent reference also changes when an influent component is perturbed. In this reduced example, the driver contains no influent term. Increasing either influent component by δ increases the required total by δ . The unweighted affine correction then adds $\delta/2$ to each predicted component. Its influent derivative is the matrix $\frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, even though the raw influent derivative is zero. This contribution comes from the right-hand side of the conservation equality.

Define the complementary projection matrices

$$Q_A = A^\top (AA^\top)^{-1} A, \quad P_A = I_F - Q_A. \quad (1.60)$$

Then $c_{aff} = P_A c_{raw} + Q_A c_{in}$. With the fitted coefficients and invariant operator held fixed, its Jacobians are

$$J_{aff,u} = P_A J_{raw,u}, \quad J_{aff,c} = P_A J_{raw,c} + Q_A. \quad (1.61)$$

The second expression includes the change in the conservation reference when the influent changes. Omitting Q_A would treat that reference as fixed and give the wrong influent sensitivity.

Multiplication by A gives $AJ_{aff,u} = 0$ and $AJ_{aff,c} = A$. An operating perturbation changes the affine prediction only along directions that preserve its fixed influent inventories. An influent perturbation changes those inventories consistently with the influent reference. These relations provide analytical checks on the sensitivity calculation.

When the affine branch is used over a neighborhood of inputs, its coefficients and Jacobians describe that branch. The complete deployment rule still includes transitions between raw, affine, and linear-program branches. A single coefficient map does not describe every branch at once.

1.7.4 Active constraints in the final correction

An explicit boundary in the same hypothetical model shows the change in sensitivity caused by nonnegativity. Keep the total influent inventory equal to 10. The affine

components obtained from Equation (1.49) are

$$c_{aff,1} = \frac{35}{6} + \frac{5a}{12}, \quad c_{aff,2} = \frac{25}{6} - \frac{5a}{12}. \quad (1.62)$$

Their sum is 10. Near $a = 10$, the first component is positive, while the second reaches zero at $a = 10$. Immediately below this boundary the affine state is feasible and its derivative is $(5/12, -5/12)^\top$. Above the boundary the affine second component is negative. The positive-weight absolute correction returns $(10, 0)^\top$, so its derivative with respect to a is $(0, 0)^\top$. The final corrected state therefore has a different derivative on each side of the boundary. A single derivative is not available at the boundary itself.

In the corrected branch, the binding equalities are $c_1 + c_2 = 10$ and $c_2 = 0$. Differentiating them gives $\partial c_1/\partial a + \partial c_2/\partial a = 0$ and $\partial c_2/\partial a = 0$. These two equations directly give the zero derivative. The general active-set calculation uses the same idea with all binding inventory and adjustment constraints.

The linear program can make a component concentration equal to zero or bind one of the absolute-adjustment inequalities. These binding constraints define an active set. If the optimal solution is unique and its nondegenerate optimal basis remains unchanged over a local neighborhood, the solution varies affinely with the relevant right-hand sides in that neighborhood. A branch-specific sensitivity can then be derived from that fixed basis.

At an active-set transition, the derivative can change abruptly. When several optimal corrected states exist, the returned solution can also depend on the rule used to select among them. A global raw-head coefficient display cannot resolve these effects. Local deployment sensitivity is assessed by differentiating a verified fixed basis where appropriate or by perturbing admissible inputs and resolving the entire deployment problem. The perturbation size and any branch changes are recorded as part of that sensitivity assessment.

This distinction prevents three different quantities from being conflated. A driver coefficient belongs to $\hat{B}$. A raw-state sensitivity passes through $\hat{R}^{-1}$. A deployed-state sensitivity also depends on the conservation reference and active correction constraints. Each quantity answers a different question about the model.

1.8 Assessment design for the structured regression

1.8.1 Common inputs and comparator models

The structured-regression assessment uses a design collection of 10,000 accepted steady-state reactor cases. Every method receives the same 22 physical inputs and predicts the same 20 effluent components. Its reported composites are obtained from the common composition matrix. The input domain, reaction model, simulation bounds, and acceptance criterion remain those of the declared reactor benchmark. The comparison

therefore changes the predictor while holding the modeled process and prediction task fixed.

The nine unconstrained comparators are XGBoost, LightGBM, CatBoost, Adaboost, random forest, support vector regression, k -nearest neighbors, partial least squares, and a fully connected multilayer perceptron. These methods represent boosted and randomized tree ensembles, kernel regression, local neighbor averaging, latent linear regression, and nonlinear neural regression. Their foundations include the ensemble construction of ?, the boosting formulations of ???? , and the latent regression of ?. The comparators have no invariant penalty, auxiliary nonnegative fitted-state matrix, or learned component-coupling equation during fitting.

The neural comparator has hidden layers with 128, 64, and 32 units and logistic activation. Dense connections map the 22 inputs through these layers to 20 component outputs. Its capacity is evaluated under the same samples and reported-composite map as the other predictors. A transparent polynomial and a neural network can therefore differ in their internal structure without changing the supervised target.

1.8.2 Training partitions and hyperparameter selection

The primary assessment uses a row-wise 80% training and 20% held-out split with random seed 42. The specified collection gives a training pool of 8000 rows and a held-out set of 2000 rows. Hyperparameter selection uses only the training pool. Half of that pool forms a 4000-row tuning subset. A further 80% and 20% split gives 3200 tuning-training rows and 800 tuning-validation rows. This is one internal validation split rather than a cross-validation design.

Each predictor receives 100 hyperparameter trials under a seeded tree-structured Parzen estimator with seed 42. No trial pruning or wall-clock timeout is imposed. ? develop this search approach, while ? provide the associated hyperparameter optimization framework. The selection criterion is mean squared error in the shared physical component space on the internal validation set. Selected settings are held fixed for the primary assessment and the repeated sample-size design. The final primary fit uses the full 8000-row training pool.

For the unconstrained comparators, input and output standardization parameters are estimated from the relevant training partition. Predictions are transformed back to physical component coordinates before any invariant check, reporting map, or error assessment. ICSOR remains in physical coordinates so that its driver, invariant operator, and deployed constraints use the same component convention. Internal-validation transformations are estimated from tuning-training rows only. Held-out observations do not determine the means, standard deviations, hyperparameters, or correction weights.

Table 1.2 gives the specified numerical configurations. These values define fitted model settings rather than measured predictive performance. The internal search compares candidate configurations according to the stated validation objective.

Table 1.2: Specified configurations of the nine unconstrained comparators.

Comparator	Configuration
XGBoost	309 trees, learning rate 0.125, maximum depth 3, minimum child weight 6.84, row fraction 0.915, feature fraction 0.828, absolute penalty 1.18×10^{-3} , and squared penalty 0.382.
LightGBM	302 trees, learning rate 0.0809, 16 leaves, maximum depth 8, minimum leaf count 14, row fraction 0.845, feature fraction 0.869, absolute penalty 3.17×10^{-3} , and squared penalty 3.36×10^{-4} .
CatBoost	236 boosting iterations, learning rate 0.172, depth 4, leaf squared penalty 0.0266, bagging temperature 0.0277, and random strength 0.0657.
AdaBoost	236 estimators, learning rate 0.999, and squared regression loss.
Random forest	390 trees, maximum depth 14, minimum split count 3, minimum leaf count 1, feature fraction 0.500, and no bootstrap resampling.
Support vector regression	Polynomial kernel of degree 4, regularization parameter 0.118, insensitive-error width 0.0458, constant offset 0.889, and solver tolerance 8.96×10^{-4} . The kernel scale is the reciprocal of the product of input dimension and training input variance.
k -nearest neighbors	15 neighbors, inverse-distance weighting, Manhattan distance, and a k -dimensional search tree with leaf size 40.
Partial least squares	6 latent components, maximum 887 fitting iterations, and tolerance 9.91×10^{-5} .
Multilayer perceptron	Hidden widths (128, 64, 32), logistic activation, adaptive moment estimation, squared penalty 6.33×10^{-4} , batch size 64, initial learning rate 1.08×10^{-3} , maximum 500 iterations, and tolerance 1.63×10^{-5} . Training rows are not shuffled, and early stopping is disabled.

1.8.3 Scoring states and physical diagnostics

The scored ICSOR output is the final checked component state. For comparator accuracy assessment, the raw component prediction receives the same equality-only affine alignment in Equation (1.43). The composition map is then applied to the aligned state. This comparison describes the accuracy of a checked structured prediction and equality-aligned comparator predictions. The comparator alignment imposes neither component nonnegativity nor the weighted linear correction.

Physical diagnostics use the direct raw states of the comparators. For ICSOR, diagnostics distinguish the raw coupled state, affine state, and final deployed state. This stage separation prevents equality alignment used during scoring from being attributed to the learner. At each stage, the violation frequency for diagnostic v is

$$f_v = \frac{100}{n} \sum_{i=1}^n \mathbf{1}\{v(c_i) > 10^{-10}\}, \quad (1.63)$$

where n is the number of assessed observations. The diagnostics v_{mc} and v_{nn} are defined in Equation (1.38). Frequencies, residual magnitudes, adjustment distances, and deployment-branch use describe different aspects of the same prediction procedure. None is treated as a substitute for predictive error.

1.8.4 Composite error measures

The differences among the five measures can first be computed for two hypothetical observations of one reported quantity. Take reference values $(2, 4)$ and predictions $(3, 2)$. Their residuals are $(1, -2)$. Squaring them gives $(1, 4)$, while taking absolute values gives $(1, 2)$. The mean squared error is $(1 + 4)/2 = 2.5$, its square root is approximately 1.5811, and the mean absolute error is $(1 + 2)/2 = 1.5$. These measures describe different summaries of the same two errors.

The absolute relative errors are $1/2$ and $2/4$, so their mean is 0.5 . The reference mean is 3 . Its total squared variation is $(2 - 3)^2 + (4 - 3)^2 = 2$. The prediction residual squared sum is 5 . Therefore the coefficient of determination is $1 - 5/2 = -1.5$. Its negative value says that the hypothetical predictions have greater squared error than assigning the reference mean to both observations. None of these example values is a performance finding.

For reported quantity $q \in \{1, 2, 3, 4\}$, define the prediction residual $e_{iq} = \hat{y}_{iq} - y_{iq}$. The five measures are mean squared error, root mean squared error, mean absolute error, mean absolute relative error, and the coefficient of determination. They are computed per reported quantity and summarized across the four quantities. The squared and absolute measures are

$$E_{2,q} = \frac{1}{n} \sum_i e_{iq}^2, \quad E_{root,q} = \sqrt{E_{2,q}}, \quad E_{1,q} = \frac{1}{n} \sum_i |e_{iq}|. \quad (1.64)$$

For strictly nonzero reference values, the relative measure is

$$E_{rel,q} = \frac{1}{n} \sum_i \frac{|e_{iq}|}{|y_{iq}|}. \quad (1.65)$$

It is reported as a ratio. Multiplication by 100 converts it to a percentage. A zero reference value makes this expression undefined, and a very small value can dominate it. Any zero-reference convention is declared explicitly before comparing predictors. A finite relative-error summary is not assigned to a target for which the unmodified expression is undefined.

The per-target coefficient of determination is

$$R_q^2 = 1 - \frac{\sum_i e_{iq}^2}{\sum_i (y_{iq} - \bar{y}_q)^2}. \quad (1.66)$$

A target with zero reference variance makes this expression undefined. A negative value is possible when the predictor has a greater squared error than the reference-mean predictor. This coefficient measures relative prediction error and is not a correlation coefficient.

The aggregate squared error pools the four composite residuals. Its root is

$$E_{root} = \left(\frac{1}{4n} \sum_{q=1}^4 \sum_{i=1}^n e_{iq}^2 \right)^{1/2}. \quad (1.67)$$

Aggregate absolute and relative errors use the corresponding mean over observations and composites. Aggregate R^2 uses the mean of the four target-specific coefficients. The composite coordinates retain their own physical units. Their pooled error is a numerical benchmark summary under that coordinate convention, rather than a single common material quantity. Per-target scores remain necessary to show which reported quantities contribute to an aggregate comparison.

Pooling before taking the square root also matters. If four hypothetical target mean squared errors are 1, 9, 1, and 9, the pooled mean squared error is 5 and its root is $\sqrt{5}$, approximately 2.2361. The arithmetic mean of the four target root errors is instead $(1 + 3 + 1 + 3)/4 = 2$. These are different summaries. Equation (1.67) specifies the first calculation.

The primary ordering measure is aggregate root mean squared error. The other measures provide complementary assessments. Squared error emphasizes larger residuals, absolute error gives average deviation, relative error emphasizes deviations in relation to the reference magnitude, and R^2 compares the squared residual with reference variation. A predictor need not receive the same ordering under all these measures.

1.8.5 Repeated sample-size assessment

The sample-size design uses

$$n \in \{500, 1450, 2400, 3350, 4300, 5250, 6200, 7150, 8100, 9050, 10000\}. \quad (1.68)$$

At each size, ten subsamples are drawn without replacement from the design collection. Deterministic seeds are derived from seed 42. Each run uses a row-wise 80% training and 20% held-out split. All ten predictors receive the same sampled rows and partitions. The design specifies 110 fits per predictor across the size schedule. This is repeated random splitting rather than ten-fold cross-validation.

The fixed hyperparameters isolate sensitivity to available observations under one model specification. The mean and spread of each held-out score are summarized across the ten runs. Learning curves are assessed over the entire schedule because behavior at one sample size does not establish behavior at other sizes. ? explain the relation between flexibility and the bias-variance problem, and ? review the varied shapes of learning curves.

The learning-curve area can first be calculated over a reduced hypothetical schedule. Suppose an error measure has values 6, 4, and 3 at sample sizes 100, 200, and 400. Between the first two sizes, a trapezoid has width 100 and mean height $(6 + 4)/2 = 5$, giving area 500. Between the next two, its width is 200 and mean height $(4 + 3)/2 = 3.5$, giving area 700. The total area is 1200. Dividing by the total width $400 - 100 = 300$ gives a normalized area of 4. A larger interval contributes more to this average than a smaller interval, even when each has one pair of endpoints. This schedule is a teaching illustration and does not replace the stated assessment sizes.

For successive retained sizes $n_1, \dots, n_S$, write the mean error of predictor m under measure j as $\overline{M}_{m,j}(n_s)$. The general numerical calculation is

$$\frac{1}{n_{max} - n_{min}} \sum_{s=1}^{S-1} (n_{s+1} - n_s) \frac{\overline{M}_{m,j}(n_s) + \overline{M}_{m,j}(n_{s+1})}{2}. \quad (1.69)$$

Let $\overline{M}_{m,j}(n)$ denote the mean held-out measure j for predictor m at sample size n . The normalized area under its learning curve is

$$L_{m,j} = \frac{1}{n_{max} - n_{min}} \int_{n_{min}}^{n_{max}} \overline{M}_{m,j}(n) \, dn. \quad (1.70)$$

The trapezoidal rule evaluates this integral over the stated sample sizes. Normalization by the sample-size interval gives an average height of the learning curve and preserves the units of measure j . It does not normalize the prediction error by a target standard deviation. ?? provide related trajectory summaries in active learning. The present design changes the number of sampled observations without choosing observations through an active acquisition rule.

A rank discards the numerical distance between scores and keeps their order. For four hypothetical predictor errors 3.0, 3.0, 4.5, and 6.0, the dense ranks are 1, 1, 2, and 3. The tied first two predictors receive the same rank. There is no skipped rank after the tie. The difference between the third and fourth errors does not influence the difference between their ranks. If one predictor receives ranks 1, 2, and 2 in three illustrative comparisons, its mean rank is $(1 + 2 + 2)/3 = 5/3$. The actual assessment averages over the stated combinations of target, measure, and size.

For each measure j , reported quantity q , and retained size n_s , predictors receive a dense rank $r_{m,j,q}(n_s)$ from their mean held-out score. Lower squared, root, absolute, and relative errors receive better ranks. Larger R^2 values receive better ranks. Equal scores receive equal ranks, and the next distinct score receives the next consecutive rank. With $S = 11$ sizes and five measures, the summaries are

$$\bar{r}_{m,j} = \frac{1}{4S} \sum_{q=1}^4 \sum_{s=1}^S r_{m,j,q}(n_s), \quad \bar{r}_m = \frac{1}{5} \sum_{j=1}^5 \bar{r}_{m,j}. \quad (1.71)$$

These are descriptive summaries of relative ordering. They do not establish statistical significance. ? develop broader rank-based model comparisons, while ? emphasize variability in benchmarking. Their contributions support careful comparative assessment without equating a mean rank with an absolute error or a significance test.

At the largest sample size, the generalization gap is

$$\Delta E_{root,m} = E_{root,m}^{held-out} - E_{root,m}^{training}. \quad (1.72)$$

A positive gap indicates greater held-out error. A small gap can accompany either accurate prediction or high-error underfitting, so it is interpreted with the corresponding absolute scores. Training time is recorded for every repeated fit and summarized as a distribution across runs. Deployment effort is assessed separately through prediction time, correction-branch use, and constrained-solve effort. This separation makes the cost of learning distinct from the cost of returning a checked state.

For a hypothetical predictor with training error 0.5 and held-out error 0.8, the gap is 0.3. Another with errors 5.0 and 5.1 has a gap of only 0.1. The second has the smaller gap but much larger absolute errors. This arithmetic explains why the gap cannot by itself identify the more useful surrogate.

1.9 Interpretability assessment and modeling limits

The interpretability assessment examines the six driver blocks, the coupling matrix, and the corresponding component and composite raw-head maps. It evaluates whether important terms remain stable across repeated training partitions and whether the associated response patterns agree with the declared process domain. Stability assessment is particularly relevant because an explicit polynomial library does not remove identifiability problems. ? connect interpretable nonlinear terms to the problem of recovering structure from data, while ? show how operational and influent uncertainty complicate parameter identification in activated sludge models.

The driver and coupling parameters can trade off in the relation $Rc = B\phi$. Different joint stationary solutions can therefore predict similar component states with different coefficients. Symmetry removes the redundancy of ordered self-quadratic pairs. It does not establish uniqueness of B and Γ together. Coefficient signs and magnitudes are

interpreted as properties of a specified fitted representation. Repeated stability checks and response comparisons provide stronger evidence than an isolated coefficient value.

Prediction error and physical diagnostics are assessed separately for the raw head, the affine state, and the final deployed state. The comparison distinguishes what the regression learns from what the correction enforces. The magnitude of the correction is also examined because a feasible deployed state can lie far from its raw prediction. Computational assessment records fitting effort and deployment effort separately. These measures establish the cost of the structured regression and the cost of enforcing its physical conditions without assuming an accuracy benefit.

The scope remains steady-state component prediction within the stated input domain and control volume. The corrected state preserves the adopted invariants and nonnegativity when the optimization and numerical checks succeed. It does not inherit the kinetic equations, dynamic behavior, or thermodynamic conditions of the full process model. The raw head supplies an inspectable approximation. The deployment constraints supply defined material-accounting conditions. Their combined use is evaluated as an engineering surrogate under these limits.